Services, Router & Reactive Programming

Web Development — Lecture 6

Seitbekov Sanzhar

KAZAKH-BRITISH TECHNICAL UNIVERSITY

KBTU · 2025 / 2026

Lecture Plan

PART 1 · 50 MIN

Architecture & Data

→ Angular Modules & NgModules

→ Angular Router — navigation basics

→ Route guards & lazy loading

→ Working with RESTful APIs

→ HttpClient — GET, POST, PUT, DELETE

PART 2 · 50 MIN

Services & Reactive Patterns

→ Angular Services & DI

→ Reactive Programming concepts

→ RxJS Observables

→ Common operators: map, filter, switchMap

→ Live coding: Student API Dashboard

Quick Recap: Where We Are

LECTURE 5

Components

→ @Component decorator, template, styles

→ Data binding — `[]`, `()`, `[()]`, `{{ }}`

→ Lifecycle hooks — ngOnInit, ngOnDestroy

→ @Input / @Output — parent ↔ child

TODAY — LECTURE 6

Beyond Components

→ How to **navigate** between pages (Router)

→ How to **fetch data** from a server (HttpClient)

→ How to **share logic** across components (Services)

→ How to **handle async streams** (Observables)

Mental model: Components are the *UI*. Services are the *brain*. The Router is the *navigation*. Observables are the *nervous system*.

01

Angular Modules & Router

Organizing your app and navigating between views

Angular Modules (NgModules)

A module is a **container** that groups related components, directives, pipes, and services.

```ts
@NgModule({
  // What this module declares
  declarations: [
    AppComponent,
    StudentListComponent
  ],
  // What this module imports
  imports: [
    BrowserModule,
    HttpClientModule,
    RouterModule
  ],
  // What services this module provides
  providers: [StudentService],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

📦

Built-in Modules

Module	Provides
BrowserModule	Core browser directives
HttpClientModule	HTTP requests
FormsModule	Two-way binding, ngModel
RouterModule	Router, routerLink

> **Angular 17+:** Standalone components are now the default — no NgModule needed for most cases. But NgModules are still common in large enterprise apps.

Angular Router — The Basics

```ts
// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: '',
    component: HomeComponent
  },
  {
    path: 'students',
    component: StudentListComponent
  },
  {
    path: 'students/:id',   // route param
    component: StudentDetailComponent
  },
  {
    path: '**',               // wildcard — 404
    component: NotFoundComponent
  }
];
```

In the template

```html
<!-- Navigation links -->
<nav>
  <a routerLink="/">Home</a>
  <a routerLink="/students">Students</a>
</nav>

<!-- Where the routed component renders -->
<router-outlet />
```

Navigating from code

```ts
constructor(private router: Router) {}

goToStudent(id: number) {
  this.router.navigate(
    ['/students', id]
  );
}
```

Reading Route Parameters

```ts
// student-detail.component.ts
import { ActivatedRoute } from '@angular/router';

@Component({ ... })
export class StudentDetailComponent
  implements OnInit {

  studentId!: number;

  constructor(
    private route: ActivatedRoute
  ) {}

  ngOnInit() {
    // Read the :id from the URL
    this.route.params.subscribe(params => {
      this.studentId = +params['id'];
    });
  }
}
```

Query Params — ?page=2&sort=name

```ts
this.route.queryParams.subscribe(
  params => {
    this.page = +params['page'] || 1;
    this.sort = params['sort'] || 'name';
  }
);
```

Note: The + before `params['id']` converts the string "42" to the number 42. Route params are always strings.

Programmatic navigation with params

```ts
this.router.navigate(['/students'], {
  queryParams: { page: 2, sort: 'name' }
});
```

Lazy Loading & Route Guards

Lazy Loading — load modules on demand

```ts
export const routes: Routes = [
  {
    path: 'admin',
    // Only loads when user visits /admin
    loadChildren: () =>
      import('./admin/admin.module')
        .then(m => m.AdminModule)
  },
  {
    path: 'dashboard',
    // Standalone component — lazy
    loadComponent: () =>
      import('./dashboard/dashboard.component')
        .then(c => c.DashboardComponent)
  }
];
```

Route Guards — control access

```ts
// auth.guard.ts
export const authGuard: CanActivateFn =
  (route, state) => {
    const auth = inject(AuthService);
    const router = inject(Router);

    if (auth.isLoggedIn()) {
      return true;
    }
    return router.createUrlTree(['/login']);
  };

// In routes:
{ path: 'admin', canActivate: [authGuard], ... }
```

Guards available: canActivate, canDeactivate (unsaved changes?), canMatch, resolve (pre-load data).

02

Working with RESTful APIs

HttpClient — fetching and sending data

REST API Basics

REST = **Re**presentational **S**tate **T**ransfer. An architectural style for building web APIs using HTTP.

HTTP Method	Action	Example URL
GET	Read / fetch	`/api/students`
POST	Create new	`/api/students`
PUT	Replace one	`/api/students/42`
PATCH	Update part	`/api/students/42`
DELETE	Remove one	`/api/students/42`

HTTP Status Codes

- **2xx** Success — 200 OK, 201 Created, 204 No Content
- **3xx** Redirect — 301 Moved, 304 Not Modified
- **4xx** Client error — 400 Bad Request, 401 Unauthorized, 404 Not Found
- **5xx** Server error — 500 Internal Server Error

JSON is the standard data format for REST APIs. Angular's HttpClient automatically parses JSON responses.

Setting Up HttpClient

1. Provide it in main.ts (Angular 17+)

```ts
// main.ts
import { bootstrapApplication } from
'@angular/platform-browser';
import { provideHttpClient } from
'@angular/common/http';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(),
    provideRouter(routes)
  ]
});
```

Or in AppModule (classic)

```ts
@NgModule({
  imports: [
    BrowserModule,
    HttpClientModule  // ← add this
  ]
})
export class AppModule {}
```

2. Inject and use in a Service

```ts
import { HttpClient } from
'@angular/common/http';

@Injectable({ providedIn: 'root' })
export class StudentService {

  private apiUrl = 'https://api.example.com';

  constructor(
    private http: HttpClient
  ) {}

  getStudents() {
    return this.http.get<Student[]>(
      `${this.apiUrl}/students`
    );
  }
}
```

HttpClient methods return Observables — not Promises. You must `subscribe()` or use the `async` pipe to get the data.

HttpClient — CRUD Operations

```ts
export class StudentService {
  // GET all
  getStudents(): Observable<Student[]> {
    return this.http.get<Student[]>
(`${this.apiUrl}/students`);
  }
  // GET one by id
  getStudent(id: number): Observable<Student> {
    return this.http.get<Student>
(`${this.apiUrl}/students/${id}`);
  }
  // POST — create
  createStudent(s: Student) {
    return this.http.post<Student>(`${this.apiUrl}/students`,
s);
  }
  // PUT — update
  updateStudent(id: number, s: Student) {
    return this.http.put<Student>
(`${this.apiUrl}/students/${id}`, s);
  }
  // DELETE
  deleteStudent(id: number) {
    return this.http.delete(`${this.apiUrl}/students/${id}`);
  }
}
```

Using it in a Component

```ts
export class StudentListComponent
  implements OnInit {

  students: Student[] = [];

  constructor(
    private svc: StudentService
  ) {}

  ngOnInit() {
    this.svc.getStudents()
      .subscribe(data => {
        this.students = data;
      });
  }
}
```

Always unsubscribe to avoid memory leaks. Use `takeUntilDestroyed()` or the `async` pipe which auto-unsubscribes.

Error Handling with HttpClient

```ts
import { catchError, throwError } from 'rxjs';
import { HttpErrorResponse } from
'@angular/common/http';

getStudents(): Observable<Student[]> {
  return this.http.get<Student[]>(this.apiUrl)
    .pipe(
      catchError((err: HttpErrorResponse) => {
        if (err.status === 404) {
          return throwError(
            () => new Error('Not found')
          );
        }
        return throwError(
          () => new Error('Server error')
        );
      })
    );
}
```

In the component

```ts
this.svc.getStudents().subscribe({
  next: data => {
    this.students = data;
    this.loading = false;
  },
  error: err => {
    this.error = err.message;
    this.loading = false;
  }
});
```

Using async pipe (recommended)

```html
<!-- No subscribe, no unsubscribe needed -->
<ul>
  <li *ngFor="let s of students$ | async">
    {{ s.name }}
  </li>
</ul>
```

03

Angular Services & Dependency Injection

Sharing logic and state across the app

What Is a Service?

A service is a plain TypeScript class decorated with @Injectable that encapsulates logic you want to **share** across multiple components.

Without services — the problem

→ Component A fetches student data

→ Component B also fetches student data

→ Two separate HTTP calls for the same data 😬

→ Logic is duplicated, hard to test

With a service — the solution

→ StudentService fetches the data once

→ Both components inject and reuse it

→ Single source of truth, easy to test ✅

TS

```ts
import { Injectable } from '@angular/core';

// providedIn: 'root' = singleton app-wide
@Injectable({
  providedIn: 'root'
})
export class StudentService {

  private students: Student[] = [];

  getAll(): Student[] {
    return this.students;
  }

  add(student: Student): void {
    this.students.push(student);
  }

  remove(id: number): void {
    this.students = this.students
      .filter(s => s.id !== id);
  }
}
```

CLI shortcut: `ng generate service student` — creates the service with the decorator and spec file.

Dependency Injection (DI)

DI is a design pattern where a class **receives** its dependencies from an external source rather than creating them itself.

```ts
// Angular's DI — inject via constructor
export class StudentListComponent {

  constructor(
    private studentSvc: StudentService,
    private router: Router,
    private http: HttpClient
  ) {
    // Angular automatically provides instances
  }
}

// Modern way — inject() function
export class StudentListComponent {
  private studentSvc = inject(StudentService);
  private router = inject(Router);
}
```

How Angular resolves DI

1. Component requests `StudentService`
2. Angular checks its injector tree
3. If singleton exists → reuse it
4. If not → create a new instance

providedIn: 'root' means Angular creates one instance shared across the whole app. This is the most common choice.

04

Reactive Programming Concepts

Thinking in streams, not single values

What Is Reactive Programming?

Reactive programming is about building systems that **react to changes** over time — streams of events, data, or errors.

Imperative (traditional)

```ts
// You pull the data when you need it
const data = fetchData(); // blocking
process(data);
```

Reactive (stream-based)

```ts
// You declare what happens when data arrives
data$.subscribe(data => process(data));
// Your code reacts to the stream
```

Real-world stream examples

→ User clicks — a stream of click events

→ Search input — a stream of keystrokes

→ HTTP responses — a stream (of 1 value)

→ WebSocket messages — continuous stream

→ Timers / intervals — a stream of ticks

Angular uses RxJS (Reactive Extensions for JavaScript) to implement reactive programming. You'll see Observables everywhere in Angular.

05

Observables in Angular

RxJS — the reactive engine powering Angular

Observable vs Promise

PROMISE

- → Represents **one** future value
- → Executes immediately on creation
- → Cannot be cancelled
- → No lazy evaluation

```ts
const p = fetchUser(1); // runs immediately
p.then(user => console.log(user));
```

OBSERVABLE

- → Represents **zero or more** future values
- → Lazy — only runs when subscribed
- → Can be cancelled (unsubscribe)
- → Composable with operators

```ts
const obs$ = fetchUser(1); // nothing runs yet
obs$.subscribe(user => console.log(user));
// now it runs
```

Convention: Variables holding Observables are named with a `$` suffix — `students$`, `search$`, `click$`. This makes them easy to spot.

Creating Observables

```ts
import { of, from, interval, fromEvent } from
'rxjs';

// of() — emit a fixed list of values
const nums$ = of(1, 2, 3);
nums$.subscribe(n => console.log(n));
// prints: 1, 2, 3

// from() — convert array or Promise to Observable
const arr$ = from(['Alice', 'Bob', 'Carol']);

// interval() — emit every N milliseconds
const timer$ = interval(1000);

// fromEvent() — DOM events as a stream
const clicks$ = fromEvent(
  document, 'click'
);
```

Subscribe with observer object

```ts
students$.subscribe({
  // Called for each value emitted
  next: (students) => {
    this.students = students;
  },
  // Called if the stream errors
  error: (err) => {
    this.error = err.message;
  },
  // Called when stream completes
  complete: () => {
    console.log('Done!');
  }
});
```

HTTP observables complete automatically after one emission. Interval observables run forever — always unsubscribe!

RxJS Operators — Transforming Streams

Operators are **pure functions** that transform the data in a stream without modifying the original Observable.

```ts
import { map, filter, tap } from 'rxjs/operators';

students$
  .pipe(
    // Transform each student object
    map(s => ({ ...s, fullName: `${s.first} ${s.last}` })),

    // Keep only passing students
    filter(s => s.grade >= 60),

    // Side effects (logging, etc.)
    tap(s => console.log('Passing:', s))
  )
  .subscribe(students => {
    this.passingStudents = students;
  });
```

Most important operators

Operator	Purpose
map	Transform each value
filter	Only pass values that match
tap	Side effects without changing stream
switchMap	Cancel previous, start new Observable
debounceTime	Wait N ms after last event
catchError	Handle errors gracefully
take	Complete after N values
combineLatest	Combine multiple streams

switchMap — The Search Use Case

switchMap cancels the previous inner Observable when a new value arrives. Perfect for search — ignore outdated results.

```ts
import { switchMap, debounceTime,
distinctUntilChanged } from 'rxjs/operators';

// Subject: manually emit values
searchQuery$ = new Subject<string>();

results$ = this.searchQuery$.pipe(
  // Wait 300ms after user stops typing
  debounceTime(300),

  // Don't search if query hasn't changed
  distinctUntilChanged(),

  // Cancel old request, start new one
  switchMap(query =>
    this.svc.search(query)
  )
);
```

Template using async pipe

```html
<input

(input)="searchQuery$.next($event.target.value
)"
  placeholder="Search students..."
/>

<ul>
  <li *ngFor="let s of results$ | async">
    {{ s.name }}
  </li>
</ul>
```

switchMap vs mergeMap vs concatMap

`switch`	Cancel previous → use for search
`merge`	Keep all concurrent → use for uploads
`concat`	Queue in order → use for saves

Subjects — Observable + Observer

A Subject is both an Observable (you can subscribe to it) and an Observer (you can push values into it). Perfect for state management in services.

```ts
import { BehaviorSubject } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class StudentService {

  // BehaviorSubject: holds current value
  private students$ = new
BehaviorSubject<Student[]>([]);

  // Expose as read-only Observable
  students = this.students$.asObservable();

  add(student: Student) {
    const current = this.students$.getValue();
    this.students$.next([...current, student]);
    // All subscribers auto-updated ✅
  }
}
```

Types of Subjects

Type	Behaviour
Subject	No initial value; no replay
BehaviorSubject	Holds current value; new subscribers get it immediately
ReplaySubject	Replays last N values to new subscribers

In the component

```ts
// Reactive — auto-updates when service
changes
students$ = this.svc.students;
```

```html
<li *ngFor="let s of students$ | async">
  {{ s.name }}
</li>
```

The async Pipe — Best Practice

The async pipe subscribes to an Observable in the template and automatically unsubscribes when the component is destroyed.

```html
<!-- Pattern 1: Direct use -->
<ul>
  <li *ngFor="let s of students$ | async">
    {{ s.name }}
  </li>
</ul>

<!-- Pattern 2: ngIf + as (subscribe once) -->
<div *ngIf="students$ | async as students">
  <p>Total: {{ students.length }}</p>
  <ul>
    <li *ngFor="let s of students">
      {{ s.name }}
    </li>
  </ul>
</div>
```

Manual vs async pipe comparison

Manual subscribe()	async pipe
Must unsubscribe manually	Auto-unsubscribes ✅
More code in component	Logic in template
Easy to forget, memory leaks	Safe by default ✅
Needed for complex logic	Best for display

Rule of thumb: Use `async` pipe when you just want to display data. Use `subscribe()` when you need to transform or store the data in a local variable.

Putting It All Together

Architecture overview

`Router`	User navigates to `/students`
`Component`	StudentListComponent activates
`Service`	StudentService injected via DI
`HTTP`	HttpClient calls REST API
`RxJS`	Observable streams data to template

Checklist for each feature

- → Define routes in `app.routes.ts`
- → Create a **Service** with `ng g service`
- → Inject `HttpClient` in the service
- → Return `Observable` from service methods
- → Inject service into component
- → Use `async` pipe in the template

Live coding today: We'll build a Student API Dashboard — full CRUD, search with debounce, and live routing between list and detail views.

Key Takeaways

🗺️

Router

- → Define routes in an array
- → Use `routerLink` in templates
- → Read params with `ActivatedRoute`
- → Guards protect routes

⚙️

Services + DI

- → `@Injectable({ providedIn: 'root' })`
- → Inject via constructor or `inject()`
- → Singleton by default
- → House your `HttpClient` calls here

🌊

Observables + RxJS

- → Lazy streams, not single values
- → Use `.pipe()` to transform
- → async pipe — safe auto-unsubscribe
- → `BehaviorSubject` for shared state

Next lecture: Angular Forms — Template-driven vs Reactive Forms, validation, and form state management.

?

Q & A

Ask anything — we'll debug it together.

How to reach me

Seitbekov Sanzhar

Lecturer

s.seitbekov@kbtu.kz

Office 178

Course chat

Microsoft Teams

Team code: **ksg13tw**

→ Announcements + materials

→ Homework questions

→ Useful links (cheatsheets, docs)